Convolution and Performance Analysis using CUDA - Lab 3

Angelos Gkertsos (03658) Nikolaos Kalamaris (03768)

1 Introduction and Device Specifications (Steps 0 & 1)

The objective of this exercise was to implement a 2D separable convolution filter on the NVIDIA CUDA architecture and analyze its scalability, precision limitations, and efficiency through optimization.

1.1 Device Specifications

The experiments were conducted using a `Tesla K80` GPU.

```
./deviceQuery Starting...

 CUDA Device Query (Runtime API) version (CUDART static linking)

Detected 4 CUDA Capable device(s)

Device 0: "Tesla K80"
  CUDA Driver Version / Runtime Version          11.4 / 11.5
  CUDA Capability Major/Minor version number:    3.7
  Total amount of global memory:                 11441 MBytes (11997020160 bytes)
  (013) Multiprocessors, (192) CUDA Cores/MP:    2496 CUDA Cores
  GPU Max Clock rate:                            824 MHz (0.82 GHz)
  Memory Clock rate:                             2505 Mhz
  Memory Bus Width:                              384-bit
  L2 Cache Size:                                 1572864 bytes
  Maximum Texture Dimension Size (x,y,z)         1D=(65536), 2D=(65536, 65536), 3D=(4096, 4096, 4096)
  Maximum Layered 1D Texture Size, (num) layers  1D=(16384), 2048 layers
  Maximum Layered 2D Texture Size, (num) layers  2D=(16384, 16384), 2048 layers
  Total amount of constant memory:               65536 bytes
  Total amount of shared memory per block:       49152 bytes
  Total shared memory per multiprocessor:        114688 bytes
  Total number of registers available per block: 65536
  Warp size:                                     32
  Maximum number of threads per multiprocessor:  2048
  Maximum number of threads per block:           1024
  Max dimension size of a thread block (x,y,z): (1024, 1024, 64)
  Max dimension size of a grid size    (x,y,z): (2147483647, 65535, 65535)
  Maximum memory pitch:                          2147483647 bytes
  Texture alignment:                             512 bytes
  Concurrent copy and kernel execution:          Yes with 2 copy engine(s)
  Run time limit on kernels:                     No
  Integrated GPU sharing Host Memory:            No
  Support host page-locked memory mapping:       Yes
  Alignment requirement for Surfaces:            Yes
```

```
  Device has ECC support:                        Enabled
  Device supports Unified Addressing (UVA):      Yes
  Device supports Managed Memory:                Yes
  Device supports Compute Preemption:            No
  Supports Cooperative Kernel Launch:            No
  Supports MultiDevice Co-op Kernel Launch:      No
  Device PCI Domain ID / Bus ID / location ID:   0 / 6 / 0
  Compute Mode:
     < Default (multiple host threads can use ::cudaSetDevice() with device simultaneously) >

Device 1: "Tesla K80"
  CUDA Driver Version / Runtime Version          11.4 / 11.5
  CUDA Capability Major/Minor version number:    3.7
  Total amount of global memory:                 11441 MBytes (11997020160 bytes)
  (013) Multiprocessors, (192) CUDA Cores/MP:    2496 CUDA Cores
  GPU Max Clock rate:                            824 MHz (0.82 GHz)
  Memory Clock rate:                             2505 Mhz
  Memory Bus Width:                              384-bit
  L2 Cache Size:                                 1572864 bytes
  Maximum Texture Dimension Size (x,y,z)         1D=(65536), 2D=(65536, 65536), 3D=(4096, 4096, 4096)
  Maximum Layered 1D Texture Size, (num) layers  1D=(16384), 2048 layers
  Maximum Layered 2D Texture Size, (num) layers  2D=(16384, 16384), 2048 layers
  Total amount of constant memory:               65536 bytes
  Total amount of shared memory per block:       49152 bytes
  Total shared memory per multiprocessor:        114688 bytes
  Total number of registers available per block: 65536
  Warp size:                                     32
  Maximum number of threads per multiprocessor:  2048
  Maximum number of threads per block:           1024
  Max dimension size of a thread block (x,y,z): (1024, 1024, 64)
  Max dimension size of a grid size    (x,y,z): (2147483647, 65535, 65535)
  Maximum memory pitch:                          2147483647 bytes
  Texture alignment:                             512 bytes
  Concurrent copy and kernel execution:          Yes with 2 copy engine(s)
  Run time limit on kernels:                     No
  Integrated GPU sharing Host Memory:            No
  Support host page-locked memory mapping:       Yes
  Alignment requirement for Surfaces:            Yes
  Device has ECC support:                        Enabled
  Device supports Unified Addressing (UVA):      Yes
  Device supports Managed Memory:                Yes
  Device supports Compute Preemption:            No
  Supports Cooperative Kernel Launch:            No
  Supports MultiDevice Co-op Kernel Launch:      No
  Device PCI Domain ID / Bus ID / location ID:   0 / 7 / 0
  Compute Mode:
     < Default (multiple host threads can use ::cudaSetDevice() with device simultaneously) >

Device 2: "Tesla K80"
  CUDA Driver Version / Runtime Version          11.4 / 11.5
  CUDA Capability Major/Minor version number:    3.7
  Total amount of global memory:                 11441 MBytes (11997020160 bytes)
  (013) Multiprocessors, (192) CUDA Cores/MP:    2496 CUDA Cores
  GPU Max Clock rate:                            824 MHz (0.82 GHz)
```

```
  Memory Clock rate:                             2505 Mhz
  Memory Bus Width:                              384-bit
  L2 Cache Size:                                 1572864 bytes
  Maximum Texture Dimension Size (x,y,z)         1D=(65536), 2D=(65536, 65536), 3D=(4096, 4096, 4096)
  Maximum Layered 1D Texture Size, (num) layers  1D=(16384), 2048 layers
  Maximum Layered 2D Texture Size, (num) layers  2D=(16384, 16384), 2048 layers
  Total amount of constant memory:               65536 bytes
  Total amount of shared memory per block:       49152 bytes
  Total shared memory per multiprocessor:        114688 bytes
  Total number of registers available per block: 65536
  Warp size:                                     32
  Maximum number of threads per multiprocessor:  2048
  Maximum number of threads per block:           1024
  Max dimension size of a thread block (x,y,z): (1024, 1024, 64)
  Max dimension size of a grid size    (x,y,z): (2147483647, 65535, 65535)
  Maximum memory pitch:                          2147483647 bytes
  Texture alignment:                             512 bytes
  Concurrent copy and kernel execution:          Yes with 2 copy engine(s)
  Run time limit on kernels:                     No
  Integrated GPU sharing Host Memory:            No
  Support host page-locked memory mapping:       Yes
  Alignment requirement for Surfaces:            Yes
  Device has ECC support:                        Enabled
  Device supports Unified Addressing (UVA):      Yes
  Device supports Managed Memory:                Yes
  Device supports Compute Preemption:            No
  Supports Cooperative Kernel Launch:            No
  Supports MultiDevice Co-op Kernel Launch:      No
  Device PCI Domain ID / Bus ID / location ID:   0 / 132 / 0
  Compute Mode:
     < Default (multiple host threads can use ::cudaSetDevice() with device simultaneously) >

Device 3: "Tesla K80"
  CUDA Driver Version / Runtime Version          11.4 / 11.5
  CUDA Capability Major/Minor version number:    3.7
  Total amount of global memory:                 11441 MBytes (11997020160 bytes)
  (013) Multiprocessors, (192) CUDA Cores/MP:    2496 CUDA Cores
  GPU Max Clock rate:                            824 MHz (0.82 GHz)
  Memory Clock rate:                             2505 Mhz
  Memory Bus Width:                              384-bit
  L2 Cache Size:                                 1572864 bytes
  Maximum Texture Dimension Size (x,y,z)         1D=(65536), 2D=(65536, 65536), 3D=(4096, 4096, 4096)
  Maximum Layered 1D Texture Size, (num) layers  1D=(16384), 2048 layers
  Maximum Layered 2D Texture Size, (num) layers  2D=(16384, 16384), 2048 layers
  Total amount of constant memory:               65536 bytes
  Total amount of shared memory per block:       49152 bytes
  Total shared memory per multiprocessor:        114688 bytes
  Total number of registers available per block: 65536
  Warp size:                                     32
  Maximum number of threads per multiprocessor:  2048
  Maximum number of threads per block:           1024
  Max dimension size of a thread block (x,y,z): (1024, 1024, 64)
  Max dimension size of a grid size    (x,y,z): (2147483647, 65535, 65535)
  Maximum memory pitch:                          2147483647 bytes
```

```
  Texture alignment:                             512 bytes
  Concurrent copy and kernel execution:          Yes with 2 copy engine(s)
  Run time limit on kernels:                     No
  Integrated GPU sharing Host Memory:            No
  Support host page-locked memory mapping:       Yes
  Alignment requirement for Surfaces:            Yes
  Device has ECC support:                        Enabled
  Device supports Unified Addressing (UVA):      Yes
  Device supports Managed Memory:                Yes
  Device supports Compute Preemption:            No
  Supports Cooperative Kernel Launch:            No
  Supports MultiDevice Co-op Kernel Launch:      No
  Device PCI Domain ID / Bus ID / location ID:   0 / 133 / 0
  Compute Mode:
     < Default (multiple host threads can use ::cudaSetDevice() with device simultaneously) >
> Peer access from Tesla K80 (GPU0) -> Tesla K80 (GPU1) : Yes
> Peer access from Tesla K80 (GPU0) -> Tesla K80 (GPU2) : No
> Peer access from Tesla K80 (GPU0) -> Tesla K80 (GPU3) : No
> Peer access from Tesla K80 (GPU1) -> Tesla K80 (GPU0) : Yes
> Peer access from Tesla K80 (GPU1) -> Tesla K80 (GPU2) : No
> Peer access from Tesla K80 (GPU1) -> Tesla K80 (GPU3) : No
> Peer access from Tesla K80 (GPU2) -> Tesla K80 (GPU0) : No
> Peer access from Tesla K80 (GPU2) -> Tesla K80 (GPU1) : No
> Peer access from Tesla K80 (GPU2) -> Tesla K80 (GPU3) : Yes
> Peer access from Tesla K80 (GPU3) -> Tesla K80 (GPU0) : No
> Peer access from Tesla K80 (GPU3) -> Tesla K80 (GPU1) : No
> Peer access from Tesla K80 (GPU3) -> Tesla K80 (GPU2) : Yes

deviceQuery, CUDA Driver = CUDART, CUDA Driver Version = 11.4, CUDA Runtime Version = 11.5, NumDevs = 4
Result = PASS
```

- **CUDA Capability:** 3.7 (Kepler Architecture).
- **Total Global Memory:** $11,441$ MB (≈ 11.99 GB).
- **Maximum Threads per Block:** 1024.

2 Implementation Strategies

2.1 Naive Single-Block Implementation (Step 2)

The initial implementation maps the entire image workload to a **single CUDA block**.

- **Indexing:** Threads use a 1D linear index (‘threadIdx.x‘) which is mathematically mapped to 2D image coordinates (x, y).
- **Limitation:** Since a CUDA block is strictly limited to 1024 threads (Tesla K80 hardware limit), this implementation supports a maximum image size of only 32×32 pixels ($32^2 = 1024$).
- **Goal:** To establish correctness and verify the convolution logic against the CPU reference.

2.2 Scalable Grid Implementation (Step 4)

To support arbitrary image sizes, we transitioned to a **Grid of Blocks** geometry.

- **Geometry:** We utilize 2D blocks of size 16×16 (256 threads) arranged in a 2D grid calculated to cover the input dimensions.
- **Indexing:** Global pixel coordinates are computed using the standard formula:

$$x = \text{blockIdx}.x \times \text{blockDim}.x + \text{threadIdx}.x$$

- **Boundary Checks:** Conditional statements ('if' checks) are required inside the kernel to prevent out-of-bounds memory accesses for image sizes that are not perfect multiples of the block size, and to handle the convolution filter apron.

2.3 Double Precision Implementation (Step 6)

This version retains the scalable grid structure of Step 4 but transitions the data types from `float` (FP32) to `double` (FP64).

- **Goal:** To quantify the trade-off between numerical precision and execution speed.
- **Hardware Implication:** Evaluating the performance penalty incurred by the Tesla K80's lower FP64 throughput and the doubled memory bandwidth requirement (8 bytes/element).

2.4 Padding Optimization Implementation (Step 8)

The final version optimizes the control flow by eliminating warp divergence.

- **Technique:** The host pre-processes the input image by adding a zero-filled "apron" (padding) of size R around the borders.
- **Benefit:** This ensures that boundary pixels have valid neighbors in memory. Consequently, the conditional boundary checks inside the critical convolution loop are removed, reducing instruction overhead and ensuring that all threads within a warp execute identical paths.

3 Scalability and Implementation Limits (Steps 3 & 4)

3.1 Thread Limit Proof (Step 3a)

The initial `Single Block` implementation was limited by the GPU's hardware capacity:

- **Maximum Supported Size:** $32 \times 32 = 1024$ threads.
- **Failure Proof:** Running the kernel with $N = 64$ (4096 threads) resulted in `invalid configuration argument`, confirming the hardware limit.

3.2 Grid Fix and Global Memory Limit (Step 4)

By implementing a multi-block `Grid` structure, the thread limit was overcome. The scalability bottleneck shifted to the available **Global Memory**.

Derivation of the Maximum Supportable Image Size (Grid Implementation)

Each Tesla K80 GPU provides 12 GB of VRAM. Due to ECC overhead, driver context, and runtime reservations, the actual accessible memory is lower. Using `cudaMemGetInfo()`, the usable VRAM is measured at approximately:

$$M_{\text{usable}} \approx 11.40 \text{ GB} = 11.40 \times 2^{30} \text{ bytes} \approx 12,240,656,793 \text{ bytes}$$

To execute the separable convolution algorithm on the GPU (Step 4), our implementation requires the concurrent allocation of three distinct arrays of size $N \times N$:

1. `d_Input`: The original input image,

2. `d_Buffer`: The intermediate result after the row convolution,

3. `d_Output`: The final result after the column convolution.

Since each element is a single-precision floating-point number (4 bytes), the total memory consumption per pixel is:

$$b_{\text{total}} = 3 \times 4 \text{ bytes} = 12 \text{ bytes}$$

The theoretical maximum dimension N is found by solving:

$$12 \cdot N^2 \leq M_{\text{usable}}$$

Substituting the available memory:

$$N \leq \sqrt{\frac{11.40 \times 2^{30}}{12}}$$

Computing the value:

$$\frac{12,240,656,793}{12} \approx 1,020,054,732$$

Taking the square root:

$$N_{\text{max, theoretical}} \approx \sqrt{1,020,054,732} \approx 31,938$$

Therefore, the theoretical upper bound for the image size is approximately $31,938 \times 31,938$. Any attempt to allocate an image significantly larger than this (e.g., $32,000 \times 32,000$) will result in a `GPU malloc failed` error, as it would require more memory than the hardware can provide.

In practice, due to memory fragmentation and slight overheads not captured by `cudaMemGetInfo`, the safe operational limit is slightly lower, typically around $N \approx 31,500$.

3.3 Accuracy Analysis: FP32 vs FP64 (Steps 5a & 6a)

We evaluated the numerical stability of the convolution kernel by comparing Single (FP32) and Double (FP64) precision implementations across varying filter radii.

3.3.1 FP32 Results (Step 5a)

The FP32 implementation suffers from significant precision loss due to the limited 24-bit mantissa. As the filter radius (R) increases, the accumulation of rounding errors leads to divergence from the CPU reference. As shown in Table 1, the kernel fails to meet the accuracy criteria for $R \geq 8$.

Table 1: FP32 Accuracy Results (Step 5a) — Image Size: 1024×1024

Filter Radius (R)	Image Size	Decimal Digits
1–2	1024	1
4	1024	0
≥ 8	1024	**Failed**

3.3.2 FP64 Results (Step 6a)

In contrast, FP64 maintains high precision (7–10 decimal digits) for all filter sizes (Table 2). **Conclusion:** Since the CUDA logic is identical in both steps, the success of the FP64 implementation proves that the algorithm is correct. The failures observed in Step 5a are strictly due to FP32 arithmetic limitations, not logic errors.

Table 2: FP64 Accuracy Results (Step 6a) — Image Size 1024×1024

Filter Radius (R)	Image Size	Decimal Digits
1	1024	10
2	1024	9
4–8	1024	8
16–32	1024	7

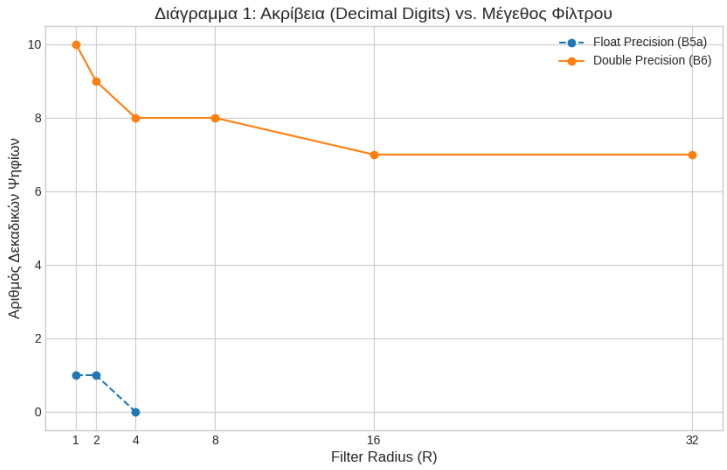

Figure 1: Accuracy comparison. FP64 ensures stability and correctness across all radii, whereas FP32 degrades rapidly for large filters.

3.4 Execution Time Scaling: CPU vs GPU (Steps 5b & 6b)

We measured the execution time for both CPU and GPU implementations across increasing image sizes, using a filter radius of $R = 16$. The GPU times include memory transfer overhead (host-to-device and device-to-host) but exclude allocation time.

3.4.1 FP32 vs FP64 Timing Results

Tables 3 and 4 present the timing measurements. The GPU achieves significant speedup over the CPU, particularly for larger images where the massive parallelism of the device is fully utilized.

Table 3: FP32 CPU/GPU Timing (Step 5b) — Filter Radius $R = 16$

Image Size	Avg CPU (ms)	Avg GPU (ms)	Speedup
64	0.394	0.127	3.10x
128	1.603	0.228	7.03x
256	6.454	0.667	9.67x
512	28.435	2.628	10.82x
1024	132.588	8.633	15.36x
2048	541.480	33.556	16.13x
4096	2337.878	143.317	16.31x
8192	9539.668	560.166	17.03x
16384	38222.393	2180.159	17.53x

Table 4: FP64 CPU/GPU Timing (Step 6b) — Filter Radius $R = 16$

Image Size	Avg CPU (ms)	Avg GPU (ms)	Speedup
64	0.461	0.153	3.01x
1024	149.990	11.375	13.18x
2048	603.030	43.311	13.92x
4096	2490.681	210.079	11.85x
8192	10076.800	830.984	12.12x
16384	41298.142	3186.564	12.96x

3.4.2 Performance Analysis

Figures 2 and 3 illustrate the scaling behavior and speedup factor respectively.

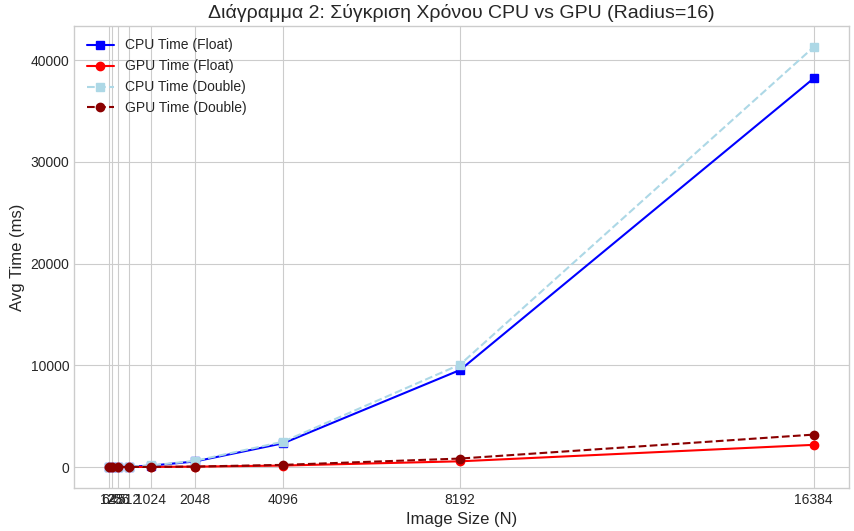

Figure 2: CPU vs GPU execution time comparison ($R = 16$). Execution time scales quadratically ($O(N^2)$) with image size, indicating the algorithm is primarily memory-bound.

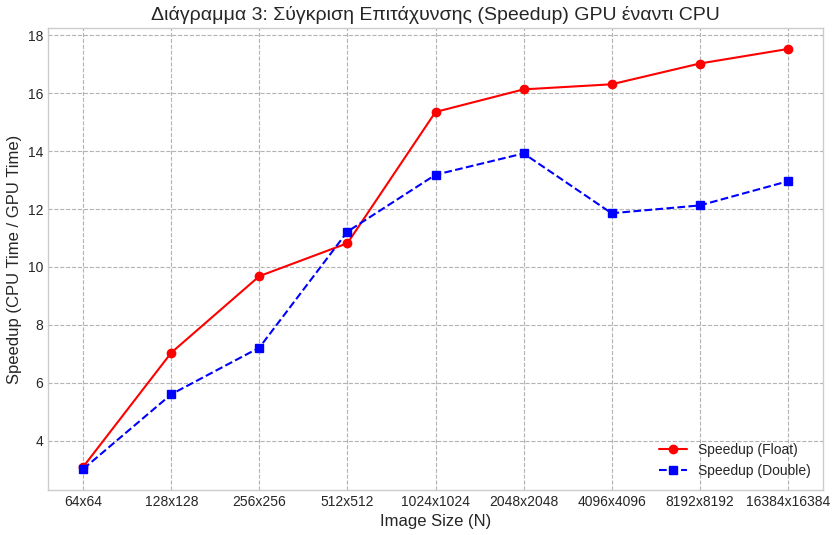

Figure 3: GPU Speedup. The GPU advantage grows with problem size, saturating at $\approx 17.5\times$ for FP32 and $\approx 13\times$ for FP64.

Key Observations:

- **Saturation of GPU Resources:** For small images ($N < 512$), the speedup is limited by initialization and PCIe transfer overheads. As N increases, the GPU's massive parallelism is saturated, stabilizing the speedup at approximately $17.5\times$ for floats.
- **Memory-Bound Scaling:** The execution time for both architectures scales linearly with the total number of pixels (N^2). This confirms that the operation is memory-bound; the bottleneck is memory bandwidth rather than arithmetic throughput.
- **FP32 vs FP64 Penalty:** On the GPU, the FP64 implementation is approximately $1.46\times$ slower than FP32 for the largest case. This is expected with the doubled memory bandwidth requirement (8 bytes vs 4 bytes per element).
- **CPU Behavior:** Interestingly, the CPU execution times between FP32 and FP64 are very similar ($\approx 8\%$ difference). This suggests that the CPU implementation is heavily bottlenecked by main memory bandwidth (RAM), preventing it from exploiting the potential SIMD advantage of single-precision floats.

3.5 Theoretical Memory and Compute Analysis (Step 7)

We analyze the memory access patterns and computational intensity of the separable convolution kernel. We consider reads from Global Memory and floating-point arithmetic operations (multiplications and additions).

3.5.1 Data Reuse Analysis (Question 7a)

To understand the access efficiency, we determine how many times each data element is retrieved from memory. We focus on the **steady-state behavior** (pixels not on the image boundaries):

- **Input Image Elements:** In a separable convolution with radius R, each pixel acts as a neighbor to $(2R + 1)$ other pixels in the same row/column. Therefore, in the general case, each element of the input image is read $(2R + 1)$ **times** per pass.

- **Filter Elements:** The filter weights are constant across the entire image. Since every thread computes one pixel and iterates through the entire filter, each element of the filter is read $N \times N$ **times** (once for every pixel in the image).

3.5.2 Compute-to-Memory Ratio (Question 7b)

We calculate the ratio of Global Memory reads to Floating-Point Operations (FLOPs) inside the kernel's inner loop.

1. Floating-Point Operations: Inside the convolution loop, for each filter tap within the image bounds, we perform:

$$1 \text{ Multiply (pixel } \times \text{ weight)} + 1 \text{ Add (accumulation)} = 2 \text{ FLOPs}$$

2. Global Memory Reads: Since both the image array (`d_Input`/`d_Buffer`) and the filter array (`d_Filter`) reside in Global Memory, each active iteration requires:

$$1 \text{ Read (Image)} + 1 \text{ Read (Filter)} = 2 \text{ Memory Reads}$$

3. The Ratio: Dividing the accesses by the operations:

$$\text{Ratio} = \frac{\text{Memory Reads}}{\text{FLOPs}} = \frac{2}{2} = 1$$

Interpretation: The kernel performs **1 Global Memory Read per 1 FLOP**. A ratio of 1:1 indicates that the algorithm is strictly **Memory-Bound**, as modern GPUs typically require a ratio of 50–100 FLOPs/byte to saturate compute units.

Note on Boundary Conditions: The above analysis represents the worst-case scenario corresponding to the majority of pixels located centrally in the image. For **boundary pixels**, the code includes a conditional check (`if`) to handle the filter radius extending beyond image limits. In these cases, the out-of-bounds iterations are skipped, resulting in fewer memory reads and fewer FLOPs. However, since both the memory accesses and the arithmetic operations are skipped simultaneously inside the conditional block, the **Compute-to-Memory Ratio remains 1:1** for the executed instructions.

3.6 Padding Optimization: Divergence Removal (Step 8)

The baseline GPU kernels (Steps 4 and 6) utilize conditional branches inside the convolution loop to handle boundary pixels. While necessary for correctness, these checks introduce two penalties:

1. **Warp Divergence:** At image edges, threads in a warp may diverge, forcing serialized execution.
2. **Instruction Overhead:** Even for central pixels where no divergence occurs, the GPU must execute the comparison instructions ($2R + 1$ times per pixel).

3.6.1 Implementation Strategy

By *padding* the input image with zeros by R pixels on each side, we eliminate the need for boundary checks entirely. The padded image has dimensions $(N+2R) \times (N+2R)$. The kernels can now perform unconditional memory accesses, restoring full SIMT efficiency and reducing the total instruction count.

The padding overhead includes:

- Memory allocation: $(N + 2R)^2 - N^2 \approx 4NR$ extra elements.
- Host-to-Device transfer of the larger padded array.

3.6.2 Performance Results

Table 5 shows the GPU execution time with padding applied (FP32, $R = 16$).

Table 5: GPU Execution Time with Padding Optimization (Step 8) — FP32, Filter Radius $R = 16$

Image Size	Baseline GPU (ms)	Padded GPU (ms)
64	0.127	0.133
128	0.228	0.250
256	0.667	0.718
512	2.628	2.576
1024	8.633	9.540
2048	33.556	35.042
4096	143.317	131.375
8192	560.166	450.913
16384	2180.159	1762.765

3.6.3 Speedup Analysis

Table 6 calculates the specific speedup factor achieved by the optimization.

Table 6: Speedup from Padding Optimization (FP32)

Image Size	Speedup	Impact Assessment
64	0.95×	Negative (Overhead Dominates)
128	0.91×	Negative
256	0.93×	Negative
512	1.02×	Neutral
1024	0.90×	Negative
2048	0.96×	Neutral
4096	1.09×	Positive
8192	1.24×	**Strong Positive**
16384	1.24×	**Strong Positive**

Observations and Interpretation

- **Small to Medium Images ($N < 4096$):** The optimization yields a net performance loss or negligible gain (Speedup $\approx 0.90 \times -1.0\times$). For these sizes, the execution time is dominated by memory transfer and kernel launch latency. The overhead of allocating and transferring the additional padding (which grows linearly with N) outweighs the computational savings inside the kernel.
- **Large Images ($N \geq 8192$):** Padding provides a substantial and consistent speedup of $\approx 1.24\times$. At this scale, the GPU is fully saturated, and the execution is limited by the aggregate instruction throughput and memory bandwidth.

  The 24% performance gain is primarily due to **Instruction Reduction**. In the baseline kernel, the loop contains an 'if' statement that is evaluated $(2R + 1)$ times for every single pixel. For $N = 16384$, this amounts to billions of comparison instructions. The padded kernel removes these instructions entirely. Even though we transfer slightly more data, the reduction in "wasted" compute cycles allows the kernel to process the data faster, leading to the observed speedup.

Conclusion Padding is an effective optimization for large-scale inputs, providing a significant $\approx 1.24\times$ speedup by streamlining the instruction pipeline. However, for smaller problem sizes, the memory overhead of the padded apron makes it counter-productive.